

Assignment 13

Generative AI Essentials: Architecture, Implementation, and Ethics

1. Introduction to Generative AI

Generative Artificial Intelligence (Generative AI) refers to machine learning systems capable of producing new content — text, images, audio, or code — that resembles the data they were trained on, rather than simply classifying or predicting a label for existing data. Its significance stems from a shift in how software is built: instead of hand-coding rules, a model learns the statistical structure of language or media and uses that structure to generate original output on demand. This capability underlies modern writing assistants, chatbots, code generators, and creative tools, and is reshaping how individuals and organizations produce content, automate communication, and prototype ideas. This report documents the architecture of Generative Pre-trained Transformer (GPT) models, a hands-on implementation of a text-generation model in Python, a demonstration of a content-creation application, and a discussion of the ethical issues raised by these systems.

2. GPT Architecture and Functionality

2.1 The Transformer and Attention

GPT models are built on the **Transformer** architecture, which replaced earlier recurrent designs by processing an entire sequence of tokens in parallel rather than one step at a time. The core mechanism is **self-attention**, which lets every token in a sequence weigh how relevant every other token is to it when building its representation. Each token is projected into a Query, Key, and Value vector; attention scores are computed from the similarity between Queries and Keys and used to blend the Values, so that words that matter most for interpreting a given word contribute the most to its updated representation. GPT uses only the **decoder** half of the original Transformer, with **masked** self-attention so a token can only attend to itself and the tokens before it — this enforces the left-to-right generation order. Stacking many such attention-and-feed-forward layers (dozens in large models) lets the network build increasingly abstract representations of grammar, meaning, and context.

2.2 Tokenization, Probabilities, and Sequence Generation

Before text reaches the network it is split into **tokens** — sub-words, whole words, or characters, depending on the tokenizer. Each token is mapped to an integer ID and then to a learned embedding vector. During training, the model repeatedly predicts the next token given the previous ones; its output layer is a softmax over the entire vocabulary, producing a **probability distribution** for what should come next. Generation is simply repeated sampling from that distribution: pick (or sample) a token, append it to the sequence, and feed the new, longer sequence back into the model. Decoding strategies control how the next token is chosen from the distribution — *greedy* decoding always takes the highest-probability token, while *temperature*, *top-k*, and *top-p (nucleus)* sampling reshape or truncate the distribution to trade off coherence against creativity, as demonstrated in Section 3.

3. Methodology and Findings

Dataset: The Tiny Shakespeare corpus (public-domain plays and poems, sourced from Project Gutenberg text commonly used for text-generation demonstrations) was used, truncated to ~120,000 characters and lower-cased for a manageable training time. **Tokenizer:** a word-level Keras Tokenizer was fit on the corpus, producing a vocabulary of 3,461 unique words. **Model:** a compact recurrent generative model was implemented in TensorFlow/Keras — an Embedding layer (64 dimensions) feeding an LSTM layer (128 units), followed by a Dense softmax layer over the vocabulary. Training examples were built with a sliding window of 6 words predicting the 7th, yielding 21,583 training sequences. The model was trained for 25 epochs (batch size 256) using the Adam optimizer and categorical cross-entropy loss.

Training loss decreased steadily from 7.14 to 5.11 and next-word accuracy rose from 3.6% to 9.6% over 25 epochs (Figure 1) — expected behavior for a small LSTM trained briefly on a modest vocabulary; a larger model, more epochs, or a pre-trained model such as GPT-2 would achieve substantially better fluency. A custom `generate_text()` function was implemented with configurable temperature, top-k, and top-p sampling (Section 4), directly reproducing the decoding controls used by production GPT systems.

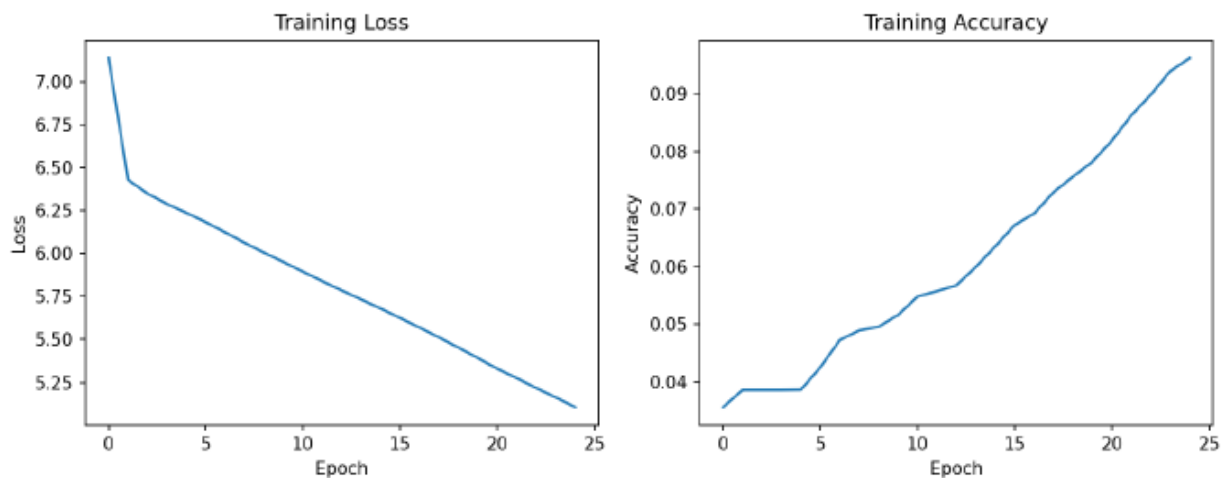


Figure 1. Training loss and accuracy over 25 epochs.

4. Applications and Practical Demonstration

Generative AI is applied today in drafting and editing text, summarization, code completion, chatbots, image and audio synthesis, and rapid prototyping of creative content. As a practical demonstration, the trained model was used as a lightweight **content-creation assistant**: given a short prompt, it produces continuation text that an author could use as a creative starting point. The exercises below show how decoding parameters change the output for the seed phrase “*to be or not to*”.

Greedy decoding (always picks the most likely word) quickly loops: “...*the people of the people of the people...*” — a well-known failure mode of deterministic decoding. **Low temperature (0.2)** stays close to this repetitive, safe behavior, while **high temperature (1.5)** produces more varied but less grammatical text (e.g. “...*my movers o' sir lo yourselves like you...*”). **Top-k=5** restricts choices to the 5 most likely words at each step, giving safer, more repetitive text than **top-k=20**. **Top-p=0.5** (narrow nucleus) is noticeably more conservative than **top-p=0.85** (wide nucleus), confirming that nucleus sampling adapts the candidate pool size to the model's confidence at each step. For the content-creation prompt “*love is a*” with temperature 0.8 and top-p 0.9, the model produced “*love is a soldier dismiss him from us but not*”.

you is our action thing that the old mangles tribunes by” — imperfect grammar (expected from a small LSTM) but thematically coherent Shakespearean vocabulary, illustrating how prompt engineering (choice of seed wording) steers the vocabulary and tone of generated text even in a simple model.

5. Ethical Considerations

- **Misinformation and hallucination:** generative models can produce fluent but false or misleading content; mitigations include human review, citing sources, and clearly labeling AI-generated content.
- **Bias and representation:** models inherit biases present in their training data (e.g. cultural, gender, or racial bias); mitigations include curated and diverse datasets, bias audits, and fine-tuning with fairness constraints.
- **Copyright and data provenance:** training on scraped text/art raises consent and attribution concerns; mitigations include licensing agreements, opt-out mechanisms, and training on public-domain or properly licensed corpora, as done in this project.
- **Misuse (deepfakes, spam, plagiarism):** the same technology that helps writers can generate disinformation or academic dishonesty; mitigations include watermarking AI output, detection tools, and institutional policy.
- **Environmental cost:** training large models consumes significant energy; mitigations include using smaller task-specific models (as in this assignment) and efficient hardware.

6. Conclusion

This project traced Generative AI from its conceptual foundations — the Transformer, self-attention, and probabilistic sequence generation — through a working, hands-on implementation: a Keras LSTM model trained on Shakespearean text, exercised with greedy decoding, length control, temperature, and top-k/top-p sampling, and applied to a simple content-creation prompt. The exercises confirmed, in a small and fully reproducible setting, the same decoding trade-offs (fluency vs. diversity) that govern production-scale GPT systems. Looking forward, replacing the custom LSTM with a pre-trained Transformer (e.g. GPT-2) and fine-tuning on domain-specific text would substantially improve fluency, while continued attention to bias, provenance, and misuse safeguards remains essential as these tools become more capable and widely deployed.

Link:

https://colab.research.google.com/drive/1bBLG3RN4_Od2au_obZMq6TfaD0c9_oeE?usp=sharing

<https://github.com/Maud20250/AI-Essentials-Architecture-Implementation-and-Ethics>